

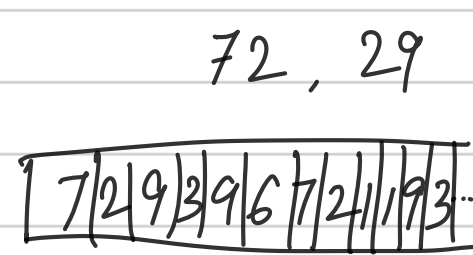
Lec 9 & Tute 3 (Large) Integer Multiplication

1/9/26

Input: Two integers x & y of n -digit.

Output: product of x & y , ie, $x \cdot y$.

(# digits of $x \cdot y$ = atmost $2n$)

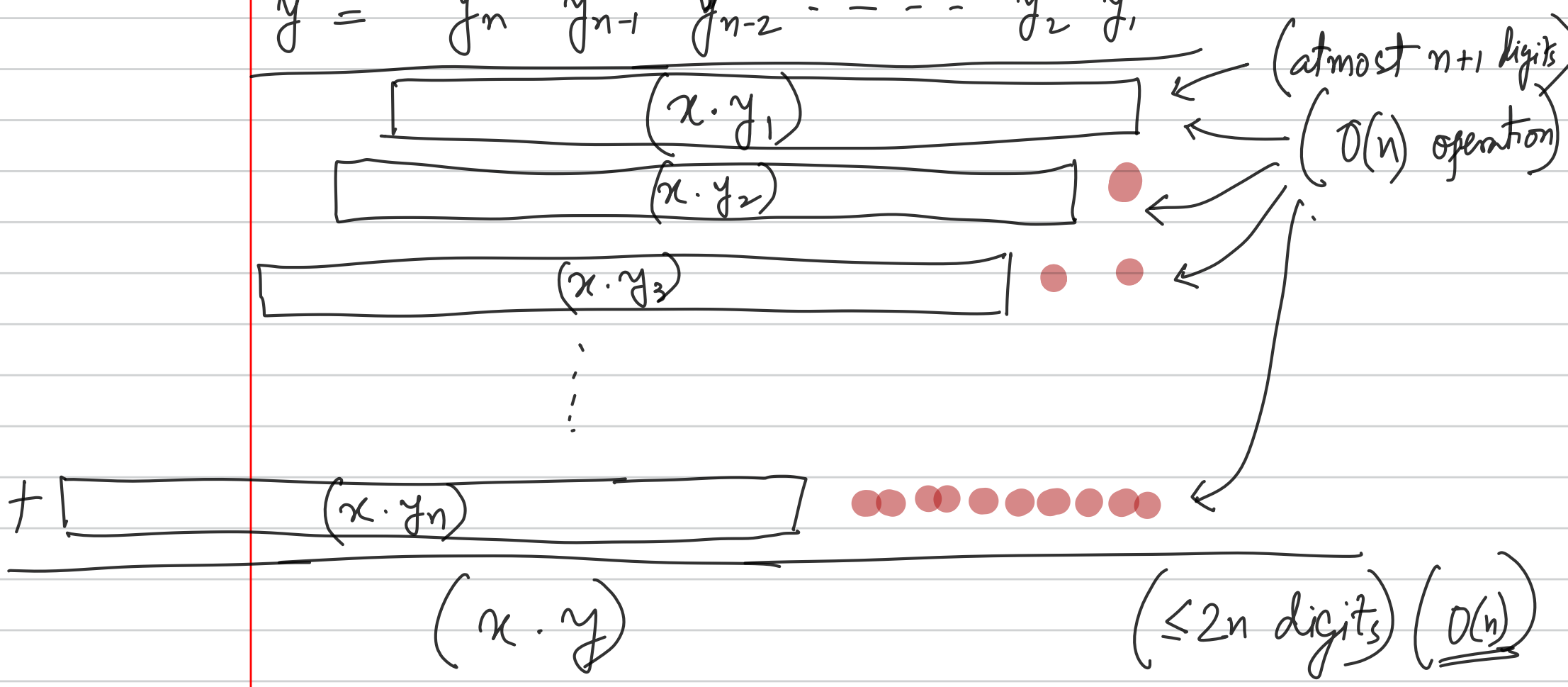


Input size for this prob is " n ".

Primitive operations as multiplication & addition of single digit.

$$x = x_n x_{n-1} x_{n-2} \dots x_2 x_1$$

$$y = y_n y_{n-1} y_{n-2} \dots y_2 y_1$$



Total running time = $O(n^2)$

Divide & Conquer tech.

$$x = \underbrace{x_n x_{n-1} \dots x_{\frac{n}{2}+1}}_a \underbrace{x_{\frac{n}{2}} \dots x_1}_b$$

$$y = \underbrace{y_n \dots y_{\frac{n}{2}+1}}_c \underbrace{y_{\frac{n}{2}} \dots y_1}_d$$

$$x = \frac{72}{a} \frac{39}{b}$$

$$x = 72 \times 100 + 39$$

$$x = (a \cdot 10^{\frac{n}{2}} + b) \parallel y = (c \cdot 10^{\frac{n}{2}} + d)$$

$$x \cdot y = \underbrace{(a \cdot c)}_{(n+1)} \cdot \underbrace{10^n}_{(n+1)} + (a \cdot d + b \cdot c) 10^{\frac{n}{2}} + bd$$

$$\underline{\underline{(a \cdot c) \times 10^n}}$$

→ only putting n 0s after (ac)
& this can be done in $O(n)$ time.

→ finding $x \cdot y$ involves 4 subprob (recursively) and other non recursive work which can be done in $O(n)$ time.

$$T(n) = 4T\left(\frac{n}{2}\right) + O(n)$$

$$a=4 \quad b=2 \quad d=1$$

$$a=4 > b^d=2$$

Case-1 $a > b^d$

$$T(n) = O(n^{\log_b a}) = \underline{\underline{O(n^2)}}$$

Karatsuba's Integer Multiplication

$$x \cdot y = \boxed{(a \cdot c)} \cdot 10^n + \boxed{(a \cdot d + b \cdot c)} 10^{\frac{n}{2}} + \boxed{bd}$$

$$(a+b)(c+d) = ac + \boxed{ad + bc} + bd$$

$$\Rightarrow ad + bc = (a+b)(c+d) - ac - bd$$

$$x \cdot y = (ac) 10^n + \left[\underbrace{(a+b)}_{\frac{n}{2}} \cdot \underbrace{(c+d)}_{\frac{n}{2}} - ac - bd \right] \cdot 10^{\frac{n}{2}} + (bd)$$

↳ 3 diff subprobs.

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n) \quad a=3 \quad b=2 \quad d=1$$

Case II $a > b^d$

$$T(n) = O\left(n^{\log_2 3}\right) = O(n^{1.58})$$

Matrix Multiplication

$$A_{n \times n} \cdot B_{n \times n}$$

$$C_{n \times n} = A \times B$$

$$\parallel$$
$$(c_{ij})$$

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj} \rightarrow O(n)$$

$\left. \begin{matrix} i \\ j \end{matrix} \right\} 1, 2, \dots, n$

$$\text{Running time} = \underline{\underline{O(n^3)}}$$

$$A_{n \times n} [b]_{n \times 1} \rightarrow O(n^2)$$

$$\begin{bmatrix} A \end{bmatrix} \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix} = \begin{bmatrix} Ab_1 & Ab_2 & \dots & Ab_n \end{bmatrix}$$

$$\begin{bmatrix} \text{---} a_1 \text{---} \\ \text{---} a_2 \text{---} \\ \text{---} a_n \text{---} \end{bmatrix} \begin{bmatrix} b_1 \end{bmatrix} = \begin{bmatrix} a_1 \cdot b_1 \\ a_2 \cdot b_1 \\ \vdots \\ a_n \cdot b_1 \end{bmatrix} \left. \begin{matrix} \rightarrow O(n) \\ \end{matrix} \right\} O(n^2)$$

$$\left(\begin{array}{c|c} [a] & [b] \\ \hline [c] & [d] \end{array} \right)_{n \times n} \quad \left(\begin{array}{c|c} [e] & [f] \\ \hline [g] & [h] \end{array} \right)_{\frac{n}{2} \times \frac{n}{2}}$$

$$\left(\begin{array}{c|c} 2 \times 2 & 2 \times 3 \\ \hline 3 \times 2 & 3 \times 3 \end{array} \right)_{5 \times 5} = \begin{array}{c} A \quad \times \quad B \\ \left(\begin{array}{c|c} [ae+bg] & [af+bh] \\ \hline [ce+dg] & [cf+dh] \end{array} \right) C \end{array}$$

We have 8 subproblems & the non recursive work can be done in $O(n^2)$

$$T(n) = 8T\left(\frac{n}{2}\right) + O(n^2) \quad \begin{matrix} a=8 & b=2 & d=2 \\ a > b^d \end{matrix}$$

Case 3 $a > b^d$

$$T(n) = O(n^{\log_2 8}) = O(n^3)$$

Strassen's Matrix Multiplication.

$$p_1 = a \cdot (f - h)$$

$$p_2 = (a + b) \cdot h$$

$$p_3 = (c + d) \cdot e$$

$$p_4 = d \cdot (g - e)$$

$$p_5 = (a + d)(e + h)$$

$$p_6 = (b - d) \cdot (g + h)$$

$$p_7 = (a - c) \cdot (e + f)$$

$$[ae + bg] \Leftrightarrow p_5 + p_4 - p_3 + p_6 \quad (\text{Verify})$$

$$[af + bh] \Leftrightarrow p_1 + p_2 \quad (\quad)$$

$$[ce + dg] \Leftrightarrow p_3 + p_4 \quad (\quad)$$

$$[cf + dh] \Leftrightarrow p_1 + p_5 - p_3 - p_7 \quad (\quad)$$

$$T(n) = 7T\left(\frac{n}{2}\right) + O(n^2)$$

$$T(n) = O\left(n^{\log_2 7}\right) = O(n^{2.8})$$

Searching Prob.

Input: An array A and an element x .

Output: Is $x \in A$??

Binary Search = ?? (when they are sorted)

(*) Even if the array is not sorted, can we solve this prob in $\Omega(n)$??

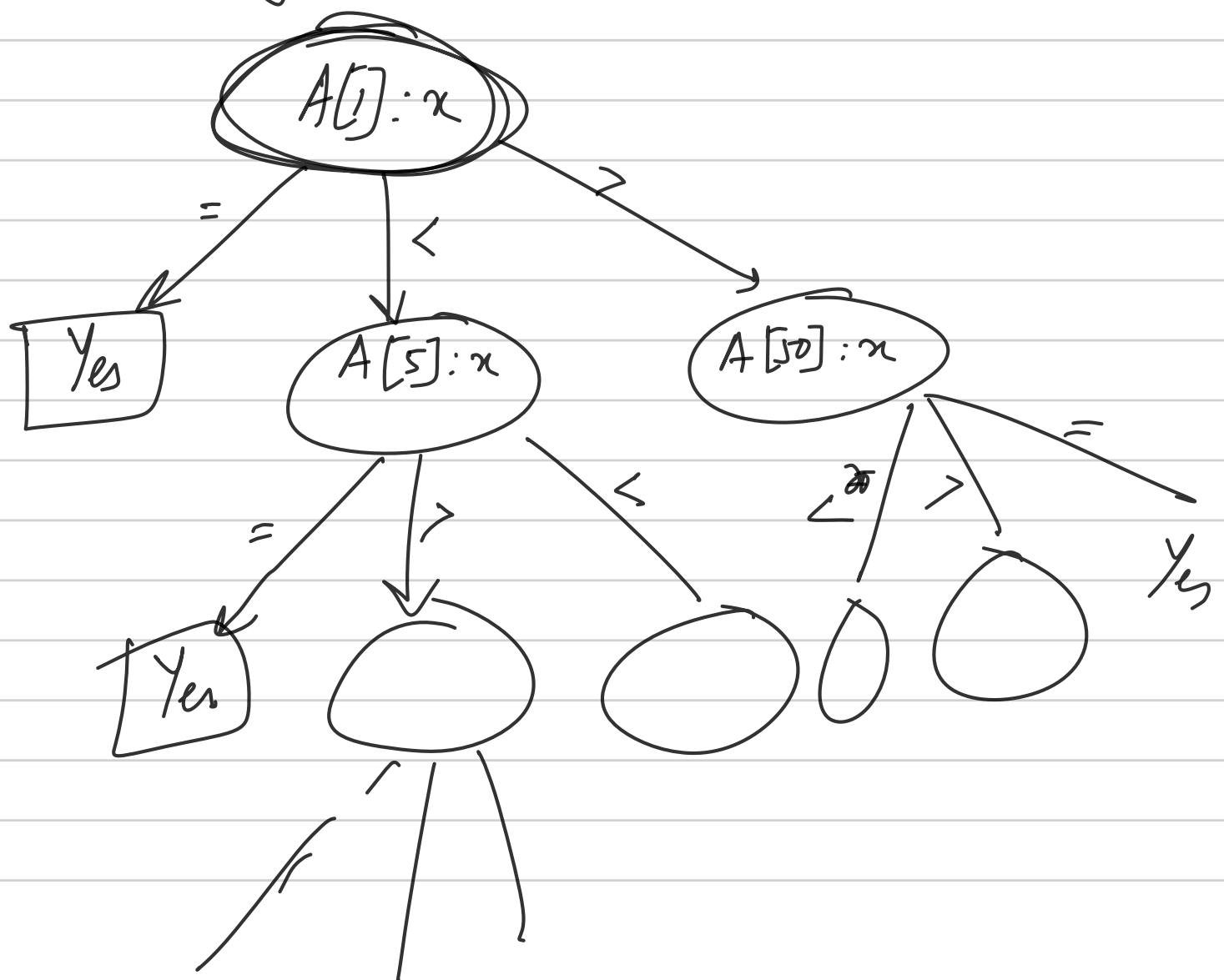
→ No, Reason :-

(*) Can we do it in $\Omega(\log n)$ time ??

No

Ex.

Prove this by decision tree.



max # leaf

$$\boxed{\underline{\underline{3^h}}} \geq \boxed{\underline{\underline{n}}}$$

possible answers.

$$\Rightarrow h \geq \log_3 n.$$